

Arquitectura del sistema y decisiones de diseño

Este documento explica *por qué* el sistema está construido como está, no qué hace (eso está en la referencia de módulos).

El problema que resuelve

Las organizaciones reciben miles de comentarios en redes sociales durante una crisis. Revisarlos manualmente es lento y subjetivo. El sistema automatiza la detección temprana: clasifica cada comentario por sentimiento y alerta cuando el nivel de negatividad supera un umbral configurable, permitiendo al equipo priorizar su atención.

Por qué la entrada es un archivo CSV (y no una API en vivo)

El sistema recibe como entrada cualquier archivo CSV con la estructura `id`, `texto`, `fecha` (la columna `usuario` es opcional). Esta decisión es deliberada:

- La API oficial de X eliminó su tier gratuito en 2023, y su costo actual (desde USD 200 mensuales en el tier Basic) la hace inviable para un proyecto académico.
- Las alternativas de scraping no autorizado fueron evaluadas y descartadas: infringen los términos de servicio de la plataforma, pueden ocasionar la suspensión de las cuentas utilizadas y no garantizan disponibilidad estable.

A cambio, el contrato de entrada es un formato universal: el sistema procesa datos de cualquier fuente que exporte CSV con esas columnas, sin modificar el código — el dataset propio del proyecto (`comentarios_x.csv`), datasets públicos de Kaggle, o una exportación futura desde una API de pago.

Para el desarrollo y la demostración se usa `comentarios_x.csv`: 672 comentarios en español que simulan publicaciones de X, contruidos a partir de casos públicos documentados (657 únicos tras la limpieza). El dataset incluye ruido deliberado (filas vacías, duplicados, símbolos, jerga) — no es una debilidad sino un dato de prueba diseñado.

Arquitectura de módulos

El sistema sigue un pipeline lineal en el que cada módulo tiene una responsabilidad única y pasa su salida al siguiente:

```
CSV de entrada
↓
[ ingesta.py ]      Lee, normaliza, deduplica, filtra ruido
↓ list[dict] con texto_limpio
[ clasificador.py ] Asigna categoría y puntuación
↓ list[dict] con categoría, puntuación, palabras_clave
[ alertas.py ]      Genera alertas individuales y globales
↓ list[dict] alertas
[ persistencia.py ] Guarda en TXT, CSV, Pickle y MySQL
↓
[ reportes.py ]     Genera reporte_crisis.txt y resumen en consola
```

`main.py` orquesta el pipeline y expone todas las operaciones a través de un menú interactivo de consola.

Cómo funciona el análisis de sentimiento

El enfoque léxico

El sistema usa un diccionario léxico ponderado en lugar de modelos de machine learning. Cada palabra del léxico tiene un peso numérico: +3 para palabras muy positivas, -3 para muy negativas, e intermedios para gradaciones.

Por qué léxico en lugar de ML: el análisis de sentimiento usa solo la biblioteca estándar de Python, sin modelos preentrenados de IA (Transformers/Hugging Face quedan explícitamente fuera del alcance). La única dependencia externa de todo el proyecto es `mysql-connector-python`, usada *exclusivamente* en la capa de persistencia y sin relación con la clasificación. Un léxico en español con 375 entradas funciona bien para comentarios de redes sociales, donde las crisis se expresan con palabras de alto impacto emocional. Además, el enfoque léxico es completamente transparente: cada puntuación es auditable palabra por palabra.

Por qué el léxico incluye jerga peruana: el dataset del proyecto proviene de casos públicos en redes sociales peruanas, donde la negatividad se expresa con coloquialismos e insultos locales que un léxico genérico no capturaría.

Intensificadores y normalización

Palabras como "muy" (+0.5) o "absolutamente" (+1.0) suman su propio peso a la puntuación bruta. Luego se normaliza por longitud: `puntuacion_normalizada = (puntuacion_bruta / n_palabras) * 10`, para tratar los comentarios por densidad negativa y no por cantidad bruta. La categorización por umbrales (`< -2.0 = muy negativo`) mitiga los extremos en textos muy cortos.

Manejo de negadores

Palabras como "no", "nunca", "jamás", "ni", "tampoco" y "sin" invierten el peso de la palabra que les sigue, y la pareja consume ambos tokens. Si el negador queda al final de la oración, no suma peso.

Por qué la deduplicación es por texto limpio

Dos comentarios son duplicados si producen el mismo texto al aplicar `limpiar_texto()`, sin importar su `id` o `fecha`. En X los usuarios copian y pegan el mismo mensaje (brigadas coordinadas); contar cada copia inflaría el porcentaje de negativos y podría disparar una alerta global falsa. La deduplicación por contenido limpio filtra esa amplificación artificial.

Diseño de las alertas

- **Alerta individual:** se dispara cuando un comentario tiene `puntuacion < umbral` (por defecto -2.0).
- **Alerta de crisis global:** se dispara cuando el porcentaje de negativos + muy negativos supera el `porcentaje_crisis` (por defecto 30%).

El campo `activa` de la tabla `alertas` (1 = activa, 0 = resuelta) permite el ciclo de vida completo con los cuatro verbos DML: se inserta al detectarse, se consulta en el monitoreo, se actualiza al resolverse y se elimina al depurar el historial.

Por qué MySQL para la persistencia

El sistema persiste su historial en una base de datos **MySQL** (la base `crisis_monitor`, creada automáticamente al iniciar). La elección responde al objetivo académico de demostrar, en vivo, un motor relacional cliente-servidor estándar de la industria: servidor dedicado, usuario y credenciales, tipos de columna explícitos (`VARCHAR`, `INT`, `TINYINT`) e índices, y los cuatro verbos DML ejecutándose sobre tablas reales durante la sustentación.

El **trade-off** es explícito y asumido: a diferencia de una base embebida en archivo, MySQL introduce una dependencia externa (`mysql-connector-python`) y requiere un servidor MySQL en ejecución con credenciales correctas. Por eso la conexión se centraliza en un único bloque `CONFIG_MYSQL` dentro de `persistencia.py`, y el resto del sistema (ingesta, clasificación, alertas, reportes) permanece independiente del motor de almacenamiento.

Por qué cuatro formatos de persistencia

Formato	Uso principal
TXT	Revisión humana directa — legible sin herramientas
CSV filtrado	Importar negativos en Excel o herramientas BI
Pickle	Restaurar el estado completo en Python para análisis posterior
MySQL	Consultas históricas, resolución de alertas, filtrado por categoría (servidor relacional, DML completo)

Cada formato tiene consumidores distintos: el CSV es para el analista de negocio; el Pickle es para el desarrollador; MySQL es para el sistema operativo con historial; el TXT es para el reporte imprimible.

Estructura del estado de sesión

```
estado = {
    "datos_analizados": None, # list[dict] | None
    "alertas": [], # list[dict]
    "estadisticas": {}, # dict
    "umbral": -2.0, # float (configurable en opción 8)
}
```

Este diseño evita variables globales sueltas y hace que cada función de menú sea explícita sobre qué datos necesita y cuáles modifica.

Manejo de errores por capa

Capa	Qué captura	Cómo lo expone
<code>ingesta.py</code>	<code>FileNotFoundError</code> , <code>ValueError</code> (columnas), <code>UnicodeDecodeError</code> (fallback a latin-1)	Propaga al llamador
<code>persistencia.py</code>	<code>mysql.connector.Error</code>	Envuelve en <code>RuntimeError</code>
<code>main.py</code> (handlers)	<code>FileNotFoundError</code> , <code>ValueError</code> , <code>RuntimeError</code> , <code>Exception</code>	Imprime el error en consola; no aborta el programa
<code>main.py</code> (bucle)	<code>KeyboardInterrupt</code> , <code>EOFError</code> , <code>Exception</code>	Ofrece confirmación de salida; continúa si se rechaza

El principio: los módulos internos propagan errores tipados; el menú los muestra en lenguaje natural y mantiene el sistema en ejecución.

Véase también: [Referencia de módulos](#) · [Tutorial de inicio](#) · [Guías de tareas comunes](#).